Práctica 3
Informática 2
Facultad de Ingeniería
Universidad de Antioquia

UNIVERSIDAD
DE ANTIOQUIA
1 8 0 3

1 Introducción

En esta práctica se implementarán algoritmos de compresión y encriptación de datos con el fin de integrar conceptos fundamentales de programación en C++. Se trabajará con los métodos RLE y LZ78 para compresión sin pérdida, así como con técnicas de rotación de bits y operación XOR para encriptación reversible. El desarrollo de esta actividad requiere el uso de arreglos, punteros, memoria dinámica y operaciones a nivel de bits. La práctica también se enfocará en las operaciones básicas de lectura y escritura de archivos de texto. Por otro lado, también se incluirá el manejo de excepciones, que proporciona un mecanismo para gestionar errores inesperados que pueden ocurrir durante la ejecución de un programa. En lugar de que el programa se cierre abruptamente, es deseable interceptar y manejar estos errores de manera controlada. Esto es crucial para crear aplicaciones robustas y confiables, asegurando que el programa pueda recuperarse de situaciones imprevistas, como la falta de un archivo o una operación que represente una indeterminación.

2 Objetivos

- Aprender a manipular cadenas de caracteres en C++.
- Aprender a gestionar archivos en C++.
- Familiarizarse con el manejo de excepciones.
- Manipular cadenas y estructuras dinámicas en el desarrollo de aplicaciones.
- Diseñar programas modulares que integren múltiples procesos sobre datos.
- Fortalecer el uso de arreglos, memoria dinámica y diseño modular en programas en C++.

3 Ejemplos

1. El profesor mostrará cómo crear un archivo y escribir en él. También cómo leer datos de un archivo.
2. El profesor mostrará cómo implementar una función básica de compresión RLE.
3. El profesor mostrará cómo realizar operaciones de rotación de bits sobre un byte.
4. El profesor realizará un ejemplo del uso de XOR para encriptar y desencriptar un carácter.
5. El profesor explicará la estructura básica de un diccionario para el algoritmo LZ78.

4 Contexto

En esta práctica se trabajará con dos procesos fundamentales en el tratamiento de la información: la compresión de datos y la encriptación. Ambos permiten transformar un mensaje original reduciendo su tamaño y modificando su representación para proteger su contenido.

4.1 Compresión de datos

La compresión sin pérdida es una técnica que permite reducir el tamaño de un mensaje garantizando que este pueda reconstruirse. En esta práctica se estudiarán dos métodos clásicos.

Run-Length Encoding (RLE)

El método Run-Length Encoding (RLE) es un algoritmo simple que aprovecha repeticiones consecutivas de un mismo símbolo. La idea consiste en reemplazar una secuencia repetida por el número de repeticiones seguido del símbolo correspondiente. Por ejemplo, la cadena:

`AAAABBBCC`

puede representarse como:

`4A3B2C`

Este método resulta eficiente cuando el texto contiene muchas repeticiones consecutivas, aunque puede no ofrecer mejoras si los datos son muy variados.

LZ78

El algoritmo **LZ78** es un método de compresión sin pérdida que construye dinámicamente un diccionario de subcadenas mientras recorre secuencialmente el texto original.

En lugar de almacenar repetidamente fragmentos que ya han aparecido, el algoritmo guarda referencias a ellos mediante índices enteros. Cada nueva entrada del diccionario se construye como la extensión de una frase previamente almacenada.

Formalmente, cada salida del algoritmo puede representarse como un par:

$$(i, c)$$

donde:

- i es el índice de una frase previamente almacenada en el diccionario (o 0 si no existe prefijo).
- c es el carácter que extiende dicha frase.

Cada nueva entrada del diccionario se define como:

$$\text{Frase}_{\text{nueva}} = \text{Frase}_i + c$$

donde Frase_0 se considera la cadena vacía.

De esta manera, el diccionario crece progresivamente reutilizando información ya almacenada.

Procedimiento general

1. Inicializar el diccionario vacío.
2. Ubicarse al inicio del texto.
3. Buscar la subcadena más larga, comenzando en la posición actual, que ya exista en el diccionario.
4. Cuando no sea posible extenderla con el siguiente carácter:
   - Emitir el par (i, c), donde i es el índice de la subcadena encontrada.
   - Agregar la nueva frase al diccionario.
5. Avanzar y continuar hasta procesar todo el texto.

Paso	Subcadena encontrada	Par emitido	Nueva entrada
1	–	$(0, A)$	$1 \to$ A
2	–	$(0, B)$	$2 \to$ B
3	A	$(1, A)$	$3 \to$ AA
4	B	$(2, A)$	$4 \to$ BA
5	BA	(4)	–

Table 1: Paso a paso del algoritmo LZ78

Ejemplo

Sea el texto:

`ABAABABA`

El proceso de compresión puede describirse paso a paso como se muestra en la Tabla 1.
Las salidas conceptuales generadas son:

$$(0, A),\ (0, B),\ (1, A),\ (2, A)$$

Durante el proceso, el diccionario evoluciona de la siguiente manera:

$$\begin{aligned} 1 &\to A \\ 2 &\to B \\ 3 &\to AA \\ 4 &\to BA \end{aligned}$$

Obsérvese que cada nueva entrada se construye a partir de una referencia previa más un carácter adicional.

Descompresión

La descompresión utiliza los pares generados durante la compresión. Para cada par (i, c):

- Si $i = 0$, la frase corresponde únicamente al carácter c.
- Si $i > 0$, la frase se obtiene concatenando la frase almacenada en la posición i del diccionario con el carácter c.

Cada frase reconstruida se agrega nuevamente al diccionario, garantizando que este crezca exactamente de la misma manera que durante la compresión.

Observación

A diferencia de métodos como RLE, que únicamente explotan repeticiones consecutivas, LZ78 identifica patrones repetidos en cualquier posición del texto. Por esta razón, puede reducir redundancia estructural incluso cuando las repeticiones no son contiguas.

4.2 Encriptación

La encriptación consiste en transformar un mensaje en otro formato con el objetivo de modificar su representación binaria. En esta práctica se utilizará un esquema sencillo basado en operaciones reversibles a nivel de bits.

Rotación de bits

La rotación de bits consiste en desplazar los bits de un byte hacia la izquierda o hacia la derecha un número determinado de posiciones. A diferencia del corrimiento tradicional, los bits que salen por un extremo se reinsertan por el extremo opuesto.
Por ejemplo, al rotar hacia la izquierda el byte:
`01000001`

se obtiene una nueva configuración binaria cuyos bits han cambiado de posición, pero conservan la misma cantidad total de 1 y 0.

`00000101`

Operación XOR

La operación XOR (eXclusive OR) es una operación lógica binaria que actúa bit a bit sobre dos valores. Su funcionamiento se basa en la siguiente regla:

Bit 1	Bit 2	Resultado (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

En palabras simples, el resultado es 1 cuando los bits son diferentes, y 0 cuando son iguales.

En programación, la operación XOR se aplica comúnmente entre dos bytes. Cuando se realiza un XOR entre dos valores de 8 bits, la operación se ejecuta de manera independiente sobre cada par de bits correspondientes.

Por ejemplo:

```
Mensaje original:   01000001   (letra 'A')
Clave (K):          01011010
--------------------------------
Resultado (XOR):    00011011
```

Cada bit del resultado se obtiene aplicando la tabla anterior posición por posición.

Propiedad fundamental

La propiedad más importante del XOR, y la razón por la cual se utiliza en encriptación simple, es la siguiente:

$$A \oplus K \oplus K = A$$

Es decir, si un valor se somete a una operación XOR con una clave K, y luego se vuelve a aplicar XOR con la misma clave, se recupera el valor original. En términos prácticos, esto significa que el mismo procedimiento puede utilizarse tanto para encriptar como para desencriptar.

En esta práctica, la encriptación de cada byte del mensaje comprimido se realizará aplicando:

1. Una rotación de bits.
2. Una operación XOR con una clave de un byte K.

Para recuperar el mensaje original será necesario aplicar nuevamente la operación XOR con la misma clave y posteriormente invertir la rotación realizada.

La simplicidad y reversibilidad de la operación XOR la convierten en un mecanismo adecuado para comprender los principios básicos de la transformación binaria de datos.

5 Ejercicios

La práctica se desarrollará en cuatro partes. Es requerido implementar y probar cada módulo de forma independiente antes de integrarlos en una sola aplicación. La primera parte, que consta de desarrollar cada uno de los módulos de compresión y encriptación, se debe realizar en la primera semana. La segunda parte, correspondiente a la integración de dichos módulos, debe realizarse en la segunda semana.

5.1 Compresión y descompresión con RLE

Implemente el algoritmo Run-Length Encoding (RLE) para comprimir y descomprimir una cadena de texto.

Para esta parte:

- Se permite el uso de la clase `std::string`.
- Se pueden utilizar operaciones propias de la clase `string` para recorrer y manipular texto.
- No se permite el uso de librerías externas de compresión.

El programa deberá:

1. Recibir una cadena de texto.
2. Generar su versión comprimida usando RLE.
3. Implementar el proceso inverso para recuperar la cadena original.
4. Verificar que la descompresión produzca exactamente el texto inicial.

Se puede usar como referencia el video Ejemplo codificación RLE

5.2 Compresión y descompresión con LZ78

Implemente el algoritmo LZ78 utilizando un diccionario dinámico construido manualmente.

Para esta parte:

- No se permite el uso de `std::vector`, `map`, `unordered_map` ni otros contenedores de la STL.
- No se permite almacenar frases completas usando `std::string`.
- El diccionario debe implementarse mediante arreglos dinámicos y memoria dinámica.
- Cada entrada del diccionario debe representarse como un par (prefijo, carácter).

El programa deberá:

1. Construir el diccionario dinámicamente durante la compresión.
2. Generar como salida pares (índice, carácter).
3. Implementar el proceso de descompresión reconstruyendo el diccionario.
4. Verificar que el texto reconstruido sea idéntico al original.

Se puede usar como referencia el video Ejemplo codificación LZ78

5.3 Encriptación y desencriptación

Implemente un módulo de encriptación que opere sobre los datos comprimidos.

El proceso deberá:

1. Aplicar una rotación de bits hacia la izquierda a cada byte, con un valor n definido por el usuario ($0 < n < 8$).
2. Aplicar posteriormente una operación XOR con una clave de un byte K.

Implemente también el proceso inverso que permita recuperar los datos originales.

5.4 Integración

Desarrolle una aplicación que permita:

- Seleccionar el método de compresión (RLE o LZ78).
- Comprimir un texto leído desde un archivo.
- Encriptar el resultado.
- Desencriptar y descomprimir el mensaje.
- Verificar que el texto final coincide con el original después de imprimirlo en un archivo.

Restricciones generales

- El uso de `std::string` está limitado únicamente a la parte del algoritmo RLE.
- En la implementación de LZ78 se deberá utilizar, de manera justificada, memoria dinámica.
- No se permite el uso de librerías externas de compresión o encriptación.
- Se solicita que el programa se organice de manera modular mediante funciones.
- Se debe aplicar manejo de excepciones dentro del código.
- Se publica junto al enunciado un dataset de prueba que pueden usar durante el desarrollo